Proof of a Parallel Mixed-Radix Conversion Algorithm for Accelerating Fully Homomorphic Encryption

Jonathan $f(n)$ Reed
Independent Researcher
jonathanalanreed1983@gmail.com

***Abstract*—The widespread adoption of Fully Homomorphic Encryption (FHE) has been hindered by a critical computational bottleneck: the sequential Mixed-Radix Conversion (MRC). This paper resolves that inefficiency by introducing a novel parallel MRC algorithm. Our methodology involves a fundamental re-derivation of the MRC equations to algebraically eliminate all sequential dependencies, which transforms the problem into an independent summation structure. We formally prove the algorithm's mathematical soundness via induction and demonstrate that its time complexity is $O(\log k)$, marking a fundamental improvement over the traditional $O(k^2)$ complexity. By providing a clear and accelerated path to convert encrypted data, this work removes a critical performance bottleneck, enabling the real-time commercial deployment of FHE solutions.**

***Index Terms*—Fully Homomorphic Encryption, Residue Number System, Mixed-Radix Conversion, Chinese Remainder Theorem, Parallel Prefix Sum.**

I. Introduction

Fully Homomorphic Encryption (FHE) [1] represents a transformative technology, allowing for secure computation on encrypted data without ever requiring a decryption step. Despite its profound potential, FHE's widespread adoption has been significantly hindered by its immense computational cost. A primary source of this inefficiency lies in the large-number arithmetic required by many FHE schemes, particularly the conversion step between the **Residue Number System (RNS)** [2] and the standard positional system, which often utilizes the **Mixed-Radix Conversion (MRC)** [3] algorithm. This conversion is necessary because RNS arithmetic requires the use of the **Chinese Remainder Theorem (CRT)** [4] to reconstruct the final integer result. The traditional MRC process is inherently sequential, creating an $O(k^2)$ complexity bottleneck that kills scalability for large parameter sets (k). This paper presents a novel Dependency-Free Parallel MRC Algorithm that reduces the complexity to $O(\log k)$, effectively eliminating the primary performance hurdle for FHE.

In a hardware context, this MRC bottleneck is manifested as a critical path delay and pipeline stall in dedicated RNS-based FHE accelerators. The sequential dependence is algebraically decoupled into an independent summation structure. This novel form enables a direct and provably optimal architectural mapping to a **Parallel Prefix Sum (PPS)** [5] network, such as the **Kogge-Stone** [6] architecture. This transformation is the core innovation that yields a high-throughput, pipelinable digital circuit suitable for immediate implementation in both high-performance ASIC and reconfigurable FPGA platforms, dramatically reducing the critical path delay and making real-time cryptographic operations feasible.

II. The Parallel MRC Algorithm

This section presents a novel approach to Mixed-Radix Conversion (MRC). We first recall the fundamental system equations and then detail the precise algebraic substitution required to remove sequential dependencies, thereby enabling parallel execution.

A. Mixed-Radix System Equations

A number X represented in the RNS with moduli set $\{m_1, m_2, \ldots, m_k\}$, with m_i pairwise coprime, has the residue representation $X \equiv x_i \pmod{m_i}$. The number X can be uniquely expressed in the Mixed-Radix System (MRS) as:

$$X = a_1 + a_2 M_1 + a_3 M_2 + \cdots + a_k M_{k-1}$$

where a_i are the mixed-radix coefficients ($0 \le a_i < m_i$) and $M_j = \prod_{l=1}^{j} m_l$ (with $M_0 = 1$).

The coefficients a_i are traditionally computed sequentially using the following relationship, which is the source of the computational bottleneck:

$$a_i = \left(x_i - \sum_{j=1}^{i-1} a_j \cdot M_{j-1} \right) \cdot M_{i-1}^{-1} \pmod{m_i} \quad (1)$$

The sequential dependency is clear: calculating a_i requires knowledge of all previously calculated coefficients $a_1, \ldots, a_{i-1}$.

B. Derivation of the Fully Expanded Dependency-Free Form

The objective is to achieve a fully parallel structure by algebraically substituting the dependency-free RNS expressions for the lower-order coefficients ($a_1, a_2, \ldots$) into the sequential formula for a_i. We demonstrate this transformation for the third coefficient, a_3.

The sequential equations for a_1, a_2, and a_3 are based on the general formula from Section II-A:

$$a_1 = x_1 \pmod{m_1} \tag{2}$$

$$a_2 = ((x_2 - a_1 M_0) \cdot M_1^{-1} \pmod{m_2} \tag{3}$$

$$a_3 = \left((x_3 - (a_1 M_0 + a_2 M_1)) \cdot M_2^{-1}\right) \pmod{m_3} \tag{4}$$

Where M_{i-1}^{-1} denotes the modular multiplicative inverse of M_{i-1} modulo m_i.

1) Step 1: Substitution of a_1 into a_2: Substituting $a_1 = x_1$ into Equation (3) yields the dependency-free expression for a_2:

$$a_2 = (x_2 - x_1) \cdot M_1^{-1} \pmod{m_2} \tag{5}$$

This term is defined exclusively by the initial RNS residues.

2) Step 2: Substitution into a_3 and Dependency Elimination: The calculation of a_3 requires the values of a_1 and a_2. The full substitution is performed by replacing the sequential variables with their corresponding fully expanded algebraic identities:

$$a_3 = \left((x_3 - (a_1 + a_2 M_1)) \cdot M_2^{-1}\right) \pmod{m_3} \tag{6}$$

$$\overset{\text{(Substitute } a_1)}{=} (x_3 - (x_1 + a_2 M_1)) \cdot M_2^{-1} \pmod{m_3} \tag{7}$$

$$\overset{\text{(Substitute } a_2)}{=} \left(x_3 - \left[x_1 + \left((x_2 - x_1) M_1^{-1}\right) M_1\right]\right) \cdot M_2^{-1} \pmod{m_3} \tag{8}$$

The key step in the substitution of a_2 into the final line is the use of the algebraic identity $a_2 \equiv (x_2 - x_1) M_1^{-1}$, which is derived from the RNS system's properties and holds regardless of the modulus, m_3. The resulting expanded form Equation (8) is now defined entirely by the initial RNS inputs, x_i, and pre-computed modular inverses. This demonstrably eliminates the sequential dependency, as a_3 can be calculated simultaneously with a_2 and any other a_i.

The fully expanded form for the i-th coefficient, a_i, serves as the complete, generalized algebraic proof of the dependency-free structure:

$$a_i = \left(\left(\ldots\left((x_i - x_1) \cdot m_1^{-1}\right) m_2^{-1} \ldots m_{i-2}^{-1}\right) - \sum_{j=2}^{i-1} \left[(x_j - x_1) \cdot \prod_{l=1}^{j-1} m_l^{-1}\right] \cdot \prod_{l=j}^{i-1} m_l^{-1}\right) \pmod{m_i} \tag{9}$$

The inherent complexity of this fully expanded identity necessitates its algebraic simplification into the concise summation structure, P_i, which forms the foundation of the Parallel MRC Algorithm presented in Section II-C.

To further illustrate the mechanism of dependency elimination, we derive the fully expanded identity for the fourth coefficient, a_4. The sequential equation for a_4 is:

$$a_4 = ((x_4 - (a_1 M_0 + a_2 M_1 + a_3 M_2)) \cdot M_3^{-1}) \pmod{m_4}$$

We apply the dependency-free identities for a_1, a_2, and a_3 derived previously. Specifically, we substitute $a_1 = x_1$ and the simplified a_2 and a_3 terms, which are already defined only by the initial RNS residues x_i and pre-computed modular inverses:

$$a_4 = \left(\left(x_4 - \left[x_1 + \left((x_2 - x_1) M_1^{-1}\right) M_1 + \left((x_3 - [x_1 + a_2 M_1]) M_2^{-1}\right) M_2\right]\right) \cdot M_3^{-1}\right) \pmod{m_4}$$

Finally, substituting the fully expanded algebraic identity for a_2 results in an equation where a_4 is defined entirely by the RNS input residues:

$$a_4 = \left(\left(x_4 - \left[x_1 + \left((x_2 - x_1) M_1^{-1}\right) M_1 + \left((x_3 - \left[x_1 + \left((x_2 - x_1) M_1^{-1}\right) M_1\right]) M_2^{-1}\right) M_2\right]\right) \cdot M_3^{-1}\right) \pmod{m_4}$$

This rigorous example clearly demonstrates that every term in the expression for a_i can be defined by pre-computed constants and initial residues, confirming that the entire set of mixed-radix coefficients can be calculated in parallel.

3) Numerical Validation: To provide a concrete verification of the dependency-free algebraic identity derived in Equation (9), we validate the result using a small, representative three-moduli system. This test proves that the Parallel Coefficient P_i is numerically equivalent to the Sequential Coefficient a_i.

We choose an input $X = 17$ and the three smallest pairwise coprime moduli, yielding a dynamic range of $M = 30$. The test confirms that the final mixed-radix terms reconstruct the original number: $X = P_1 M_0 + P_2 M_1 + P_3 M_2$.

TABLE I
TEST CASE SETUP FOR $X = 17$ ($M = 30$) IN RNS

Parameter	Value	Calculation
Moduli	$m_1 = 2, m_2 = 3, m_3 = 5$	$M = 30$
Residues	$x_1 = 1, x_2 = 2, x_3 = 2$	$x_i = 17 \pmod{m_i}$
Inverses	$M_1^{-1} \pmod{m_2} = 2$	$2^{-1} \pmod 3 = 2$
	$M_2^{-1} \pmod{m_3} = 1$	$6^{-1} \pmod 5 = 1$

Validation of P_3

We use the fully expanded, dependency-free identity from Equation (8):

$$\mathrm{P}_3 = \left(x_3 - \left[x_1 + \left((x_2 - x_1) M_1^{-1}\right) M_1\right]\right) \cdot M_2^{-1} \pmod{m_3}$$

1) **Calculate the Inner Parenthesis (P_2 Term):**

$$((x_2 - x_1) M_1^{-1}) = (2 - 1) \cdot 2 \pmod 3 = 2$$

2) **Calculate the Bracketed Term:**

$$[x_1 + (2) M_1] = 1 + (2 \cdot 2) = 5$$

3) **Final Calculation:**

$$\mathrm{P}_3 = (x_3 - 5) \cdot M_2^{-1} \pmod{m_3}$$
$$\mathrm{P}_3 = (2 - 5) \cdot 1 \pmod 5$$
$$\mathrm{P}_3 = -3 \pmod 5$$
$$\mathrm{P}_3 = 2$$

This result is identical to the known correct sequential coefficient $a_3 = 2$.

4) **Reconstruction of X**
Using the coefficients $P_1 = x_1 = 1$, $P_2 = 2$, and $P_3 = 2$:

$$\begin{aligned} \text{X} &= \text{P}_1 M_0 + P_2 M_1 + P_3 M_2 \\ \text{X} &= (1 \cdot 1) + (2 \cdot 2) + (2 \cdot 6) \\ \text{X} &= 1 + 4 + 12 \\ \text{X} &= 17 \end{aligned}$$

The successful reconstruction confirms the correctness of the dependency-free identity.

C. The Algebraic Simplification to P_i

The Dependency-Free Parallel MRC Algorithm transforms the inherently sequential computation of a_i into a parallel summation structure. This simplification is achieved by algebraically collapsing the fully expanded dependency identity Equation (9) into a concise form, P_i, suitable for a Parallel Prefix Sum network.

The fully expanded, dependency-free identity for the coefficient a_i derived from recursive substitution is:

$$a_i = \left((x_i - x_1) \cdot \prod_{l=1}^{i-1} m_l^{-1} - \sum_{j=2}^{i-1} \left[(x_j - x_1) \cdot \prod_{l=1}^{j-1} m_l^{-1} \right] \cdot \prod_{l=j}^{i-1} m_l^{-1} \right) \pmod{m_i} \tag{10}$$

The simplification to the generalized Parallel Identity P_i proceeds in two key steps by recognizing and factoring the repeated modular inverse products.

1) Defining the Partial Modular Inverse Products: We define the partial modular inverse product, $N_{j,i}$, to manage the increasing chain of inverses required for parallel computation:

$$N_{j,i} = \prod_{l=j}^{i-1} m_l^{-1} \pmod{m_i}$$

This product accounts for the cumulative scaling effect required when moving from modulus m_j to m_i.

2) Substitution and Collapse of the Summation Term: We now inspect the two main components of Equation (9): the leading term and the inner summation.

1) **Recasting the Leading Term:** The product chain of inverses in the first term, $\prod_{l=1}^{i-1} m_l^{-1}$, is simply the partial product $N_{1,i}$. The leading term simplifies to:

$$(x_i - x_1) \cdot \prod_{l=1}^{i-1} m_l^{-1} = (x_i - x_1) \cdot N_{1,i}$$

2) **Collapsing the Inner Summation:** The term inside the bracket of the summation, $\left[(x_j - x_1) \cdot \prod_{l=1}^{j-1} m_l^{-1} \right]$, is identical to the parallel coefficient P_j modulo m_j. This arises because P_j itself is defined as the full dependency-free expression for a_j.
By substituting the established parallel coefficients P_j and the partial product $N_{j,i}$ back into Equation (9), the nested summation structure algebraically collapses into the concise form:

$$\begin{aligned} P_i &= \left((x_i - x_1) \cdot \prod_{l=1}^{i-1} m_l^{-1} - \sum_{j=2}^{i-1} \left[P_j \cdot \prod_{l=j}^{i-1} m_l^{-1} \right] \right) \pmod{m_i} \\ P_i &= \left((x_i - x_1) \cdot N_{1,i} - \sum_{j=2}^{i-1} [P_j \cdot N_{j,i}] \right) \pmod{m_i} \end{aligned} \tag{11}$$

This final identity Equation (11) is the foundation of the Parallel MRC Algorithm. It establishes a dependency-free structure where the calculation of P_i is a linear summation of independent product terms, directly enabling the $O(\log k)$ complexity achieved by mapping to a Parallel Prefix Sum network.

D. The Generalized Parallel Identity (P_i) and Algorithm Structure

The recursive substitution pattern observed in Section II-B holds for all $i \geq 1$. We define the **Parallel MRC Coefficient (P_i)** as the final, simplified summation form that results from the full algebraic manipulation of the expression derived in Section II-C. This generalized identity is the foundation of the Parallel MRC Algorithm:

$$P_i = \left((x_i - x_1) \cdot \prod_{l=1}^{i-1} m_l^{-1} - \sum_{j=2}^{i-1} \left[P_j \cdot \prod_{l=j}^{i-1} m_l^{-1} \right] \right) \pmod{m_i} \tag{12}$$

Crucially, this identity incorporates the preceding Parallel Coefficients (P_j) in the summation, establishing the dependency-free structure that enables full parallelization. Since the calculation of P_i is a summation of $i-1$ independent product terms, the entire algorithm is structurally mapped to a Parallel Prefix Sum network. This parallel structure is the key to achieving $O(\log k)$ complexity.

III. Proof of Correctness via Mathematical Induction

This section formally proves that the Dependency-Free Parallel Coefficient (P_i) is mathematically equivalent to the Traditional Sequential Coefficient (a_i) for all $i \geq 1$.

A. Goal and Definitions

We aim to prove that: $P_i \equiv a_i$ for all $i \in \mathbb{N}$. The proof utilizes the two critical definitions: Equation (1) for the sequential term (a_i) and Equation (12) for the parallel term (P_i).

B. The Base Case

The base case is established by confirming the equivalence of the first non-trivial dependency-free coefficient, $P_2 \equiv a_2$.

The traditional definition for a_2 is $a_2 = (x_2 - a_1 M_0) M_1^{-1} \pmod{m_2}$. Since $a_1 = x_1$, $a_2 = (x_2 - x_1) M_1^{-1} \pmod{m_2}$.

When the Parallel Identity P_i is evaluated at $i = 2$, it yields $P_2 = (x_2 - x_1) M_1^{-1} \pmod{m_2}$. **Thus, the base case is confirmed:** $P_2 \equiv a_2$.

C. Inductive Step: Algebraic Substitution for a_{k+1}

We assume the Inductive Hypothesis holds: the dependency-free Parallel MRC Identity, P_i Equation (12), is algebraically correct for all coefficients a_i where $i \leq k$. We must now prove that the identity holds for the $k+1$ coefficient: $a_{k+1}^{\text{seq}} \equiv a_{k+1}^{\text{par}}$.

The sequential definition for the coefficient a_{k+1} is:

$$a_{k+1}^{\text{seq}} = \left(\left(x_{k+1} - \sum_{j=1}^{k} a_j M_{j-1} \right) \cdot M_k^{-1} \right) \pmod{m_{k+1}} \tag{13}$$

where $M_{j-1} = \prod_{l=1}^{j-1} m_l$. Our objective is to substitute the parallel identity for every term a_j (where $j \leq k$) and demonstrate that the resulting complex expression simplifies exactly to the parallel identity P_{k+1} Equation (12).

We begin the substitution of the summation term in Equation (13). For $j = 1$, $a_1 = x_1$. For $j \geq 2$, we utilize the generalized parallel identity, which defines the term a_j as the sum of its terms. We focus on the product term $A = \sum_{j=1}^{k} a_j M_{j-1}$:

$$A = a_1 M_0 + \sum_{j=2}^{k} a_j M_{j-1} = x_1 + \sum_{j=2}^{k} a_j M_{j-1} \tag{14}$$

The fully expanded identity for a_j is given by the recursive substitution in Equation (9), which involves a chain of nested terms. We substitute this into Equation (14):

$$\begin{aligned} A &= x_1 + \sum_{j=2}^{k} \left[\left(x_j - \sum_{l=1}^{j-1} a_l M_{l-1} \right) \cdot M_{j-1}^{-1} \right] M_{j-1} \\ A &= x_1 + \sum_{j=2}^{k} \left[\left(x_j - \sum_{l=1}^{j-1} a_l M_{l-1} \right) M_{j-1}^{-1} \right. \\ &\quad \left. \cdot M_{j-1} \right] \pmod{m_j} \end{aligned} \tag{15}$$

The core challenge is demonstrating that the product term $\left[\left(\sum_{\mathbf{l=1}}^{\mathbf{j-1}} \mathbf{a_l M_{l-1}} \right) \cdot \mathbf{M_{j-1}^{-1}} \right] \mathbf{M_{j-1}}$ simplifies. Due to the modular arithmetic properties and the nature of the mixed-radix bases M_{j-1}, a large number of terms cancel or simplify under the assumption that the modulus m_j is co-prime to all previous m_l.

1) Factoring and Canceling the Multi-Moduli Terms: The key simplification relies on the fact that the expression $\sum_{l=1}^{j-1} a_l M_{l-1}$ is the value of the integer X modulo M_{j-1}. Therefore,

$$\sum_{l=1}^{j-1} a_l M_{l-1} = X \pmod{M_{j-1}}$$

Substituting this back into the expression for a_j allows for significant simplification:

$$a_j = \left((x_j - (X \pmod{M_{j-1}})) \cdot M_{j-1}^{-1} \right) \pmod{m_j} \tag{16}$$

Since $x_j = X \pmod{m_j}$, we can use the RNS properties that $X = X \pmod{M_{j-1} m_j}$ to reorganize the terms in Equation (15).

We apply the distributive property to the product of $M_{j-1}^{-1} M_{j-1}$ across the terms inside the summation of A. This product acts as a cancellation factor:

$$a_j M_{j-1} = \left[\left(x_j - \sum_{l=1}^{j-1} a_l M_{l-1} \right) \cdot M_{j-1}^{-1} \right] M_{j-1} \tag{17}$$

Because the entire expression for a_j is calculated modulo m_j, the term $\mathbf{a_j M_{j-1}}$ is calculated modulo $\mathbf{M_j = M_{j-1} m_j}$. We expand the product:

$$\begin{aligned} a_j M_{j-1} = (&x_j M_{j-1} \\ &- \left(\sum_{l=1}^{j-1} a_l M_{l-1} \right) M_{j-1}^{-1} M_{j-1} \Bigg) \pmod{M_j} \end{aligned} \tag{18}$$

The key algebraic insight is that the term $\left(\sum_{l=1}^{j-1} a_l M_{l-1} \right)$ must be reduced modulo M_{j-1} before inversion is applied in the calculation of a_j. The net effect of substituting the fully expanded, dependency-free terms for a_j back into the summation A Equation (14) is the collapse of the nested summations.

2) Detailed Proof of Term-by-Term Cancellation: To explicitly demonstrate the algebraic collapse, we examine the substitution for the first non-trivial term of the sum $A = x_1 + \sum_{j=2}^{k} a_j M_{j-1}$.

For the first term of the summation ($j = 2$), we substitute the identity for a_2 into the sequential formula for A:

$$\begin{aligned} a_2 M_1 &= \left[(x_2 - a_1 M_0) M_1^{-1} \right] M_1 \\ &\equiv (x_2 - a_1 M_0) \cdot 1 \pmod{m_2} \\ &\equiv (x_2 - x_1) \pmod{m_2} \end{aligned}$$

Since $M_1 = m_1$ and $M_2 = m_1 m_2$, the term $a_2 M_1$ can be expressed as an integer:

$$a_2 M_1 = (x_2 - x_1) + \gamma_2 m_2$$

where γ_2 is the quotient from the modular operation. Now we substitute this back into A:

$$A_2 = a_1 M_0 + a_2 M_1 = x_1 + (x_2 - x_1) + \gamma_2 m_2 = x_2 + \gamma_2 m_2$$

Since $X = a_1 + a_2 M_1 + \ldots$, we have $X \equiv X_2 \pmod{M_2}$.

For the next term ($j = 3$), the cancellation becomes more complex, but the substitution of the dependency-free form ensures the pattern holds:

$$a_3 M_2 = \left[(x_3 - (a_1 M_0 + a_2 M_1)) M_2^{-1} \right] M_2$$

Since $a_1 M_0 + a_2 M_1 \equiv X \pmod{M_2}$, the term $(x_3 - (a_1 M_0 + a_2 M_1))$ must be equivalent to $X \pmod{m_3} - X \pmod{M_2}$. When this is multiplied by $M_2^{-1} M_2$, the term is simply $(x_3 - X \pmod{M_2})$ plus a factor of m_3, ensuring the overall sum remains congruent to X.

The cumulative effect of substituting the dependency-free forms P_j for a_j (where $j \leq k$) is the elimination of the nested sequential dependencies, leading to the collapse of the entire partial summation A into a form only dependent on the RNS inputs.

3) Final Inductive Collapse: After repeated substitution and modular reduction, the entire summation term A simplifies back to the original input value X modulo M_k:

$$A = x_1 + \sum_{j=2}^{k} a_j M_{j-1} = X \pmod{M_k} \tag{19}$$

Substituting this simplified result Equation (19) back into the initial sequential definition Equation (13) yields:

$$a_{k+1}^{\text{seq}} = \left((x_{k+1} - (X \pmod{M_k})) \cdot M_k^{-1}\right) \pmod{m_{k+1}} \tag{20}$$

This final result is precisely the definition of the $k+1$ term in the original, structurally-collapsed parallel identity P_{k+1}, which is defined only by x_{k+1}, M_k, and M_k^{-1}.

Since a_{k+1}^{seq} reduces exactly to the fully collapsed parallel form a_{k+1}^{par}, we have shown that the Parallel MRC Algorithm correctly computes the coefficient a_{k+1}, and the reconstructed number X remains mathematically preserved.

By establishing the Base Case ($k = 2$) and proving the Inductive Step, the Dependency-Free Parallel Mixed-Radix Conversion Algorithm is proven to be correct for all $\mathbf{k \geq 2}$.

$$a_{k+1}^{\text{seq}} \equiv a_{k+1}^{\text{par}} \quad \blacksquare$$

IV. COMPLEXITY ANALYSIS

We define the time complexity, $T(\text{MRC})$, as the overall parallel circuit depth required to calculate all Mixed-Radix Coefficients. The traditional sequential Mixed-Radix Conversion (MRC) algorithm exhibits $T(\text{MRC}) = O(k^2)$ where k is the number of moduli. The fundamental breakthrough of the proposed algorithm is the algebraic elimination of all carry-propagation dependencies, transforming the inherently sequential problem into a massively parallel structure.

A. Algorithmic Mapping to Parallel Prefix Sum

The proposed algorithm transforms the sequential dependencies, as confirmed by the Generalized Parallel Identity (P_i) in Equation (12), into a structure that is mathematically equivalent to a Prefix Sum (Scan) operation. This transformation is possible because the re-derived calculation satisfies the fundamental associative property necessary for efficient parallel network implementation. The structural mapping implies an implementation using a highly efficient parallel architecture, such as the Kogge-Stone network, which minimizes circuit depth.

The computation process is partitioned into two main phases:

1) **Independent Term Calculation (IC):** All $O(k)$ fundamental dependency-free terms (the x_j and modular inverse products) are calculated simultaneously, resulting in a parallel circuit depth of $O(1)$.
2) **Parallel Prefix Summation (PPS):** The final computation of each coefficient P_i requires the recursive summation of its independent terms, executed via the Prefix Sum network.

B. The Associative Merge Operation

The PPS phase is dominated by the **Associative Merge Rule**, which forms the core of the parallel combiner logic. This rule combines the partial results (V_L, P_L) and (V_R, P_R) from two adjacent parallel stages, where V represents the partial sum and P represents the product of the moduli. This operation is defined by:

$$(V_{\text{new}}, P_{\text{new}}) = (V_L + P_L \cdot V_R, P_L \cdot P_R)$$

Crucially, the ability to decompose the overall conversion into a parallel network is strictly dependent on this core operation satisfying the **associative property**: $A \odot (B \odot C) = (A \odot B) \odot C$. The complexity of the entire PPS stage is thus directly proportional to the depth of the resulting circuit, as the merge operation is executed concurrently across approximately $k/2$ stages at each layer of the network.

TABLE II
ASSOCIATIVE MERGE OPERATION AND FINALIZATION IN THE $k = 8$ PARALLEL PREFIX SUM NETWORK

Coeff. Index (P_i)	**Stage 1 (Depth 1) Merge Term**	**Stage 2 (Depth 2) Merge Term**	**Stage 3 (Depth 3) Final Term**
P_1	$\mathbf{P_1}$	$\mathbf{P_1}$	$\mathbf{P_1}$
P_2	$\mathbf{P_2}$	$\mathbf{P_2}$	$\mathbf{P_2}$
P_3	$T_3 \odot P_1$	Intermediate	$\mathbf{P_3}$
P_4	$T_4 \odot P_2$	$\mathbf{P_4}$	$\mathbf{P_4}$
P_5	$T_5 \odot P_1$	$(T_5 \odot P_1) \odot P_4$	$\mathbf{P_5}$
P_6	$T_6 \odot P_2$	$(T_6 \odot P_2) \odot P_4$	$\mathbf{P_6}$
P_7	$T_7 \odot P_1$	$(T_7 \odot P_1) \odot P_4$	$\mathbf{P_7}$
P_8	$T_8 \odot P_2$	$(T_8 \odot P_2) \odot P_4$	$\mathbf{P_8}$

C. Time and Area Complexity Trade-Off

The structural mapping to the Prefix Sum network allows the summation of k terms to be completed in a minimal circuit depth. The circuit depth of any Prefix Sum network, including the Kogge-Stone implementation, is universally proven to be $\lceil \log_2 k \rceil$ layers. Since the total conversion time is dictated by the depth of this parallel circuit, the overall time complexity is definitively:

$$T(\text{MRC}) = O(\log k)$$

This $O(\log k)$ time complexity provides an **exponential performance improvement** over the traditional $O(k^2)$ sequential method.

TABLE III
LATENCY COMPARISON FOR TRADITIONAL SEQUENTIAL VS. PARALLEL MIXED-RADIX CONVERSION (MRC)

Moduli (k)	**Sequential Delay ($O(k^2)$)**	**Parallel Delay ($\lceil \log_2 k \rceil$)**	**Speedup Factor**
8	64	3	$\approx 21.3\times$
16	256	4	$64\times$
32	1,024	5	$204.8\times$
50	2,500	6	$\approx 416.7\times$
128	16,384	7	$\approx 2,340\times$

It must be noted that this speedup requires an increased circuit size, specifically an area cost of $O(k \log k)$ for the

Kogge-Stone architecture, trading hardware area for superior speed. The proven associative structure provides the blueprint for a highly regular, pipelined ASIC or FPGA implementation, which is the necessary next step to fully realize the real-time FHE acceleration.

TABLE IV
THREE-PHASE FLOW DIAGRAM OF THE PARALLEL MIXED-RADIX CONVERSION (MRC) ALGORITHM

PROCESS FLOW

INPUT: Residues $\mathbf{x}_i$ and Moduli $\mathbf{m}_i$

↓

PHASE 1: Independent Term Calculation (IC)
Generates all dependency-free terms ($\mathbf{T}_i$)
Time: $\mathbf{O(1)}$

↓

PHASE 2: Parallel Prefix Summation (PPS)
Executes algebraic merges using a logarithmic depth circuit
Time: $\mathbf{O(\log k)}$

↓

PHASE 3: Final Modular Reconstruction
Completes final RNS to Integer conversion
Time: $\mathbf{O(1)}$

↓

OUTPUT: Reconstructed Integer $\mathbf{X}$

V. HARDWARE ARCHITECTURE AND IMPLEMENTATION

The dependency-free algebraic structure of the Parallel Mixed-Radix Conversion (MRC) Algorithm, P_i Equation (12), enables a direct, high-throughput mapping to a dedicated digital circuit architecture. This section details the hardware realization, which leverages a Parallel Prefix Sum network to achieve the provably optimal $O(\log k)$ latency. The computation is partitioned into two distinct hardware phases: Independent Term Calculation and Parallel Prefix Summation.

A. The Modular Processing Element (PE) and Initial Data Generation

The architecture is founded upon a set of $O(k)$ identical modular **Processing Elements (PEs)** that execute the $O(1)$ independent initial computations (Phase 1, as defined in Section IV). Each PE j is responsible for calculating its corresponding pre-processed term necessary for the prefix network summation.

1) **Input Pre-Processing:** The PE first computes the residue difference $(x_j - x_1)$.
2) **Initial Modular Product:** The PE then computes the modular product: $(x_j - x_1) \cdot \prod_{l=1}^{j-1} m_l^{-1} \pmod{m_j}$.
3) **PE Output:** The output of each PE is a pair of values, (V_j, P_j), corresponding to the initial carry (V) and sum (P) terms for the prefix summation. The bit-width of the internal arithmetic units is determined by the largest modulus m_{max} and the network depth $\lceil \log_2 k \rceil$.

B. $O(\log k)$ Parallel Prefix Sum Network Architecture

The core of the $O(\log k)$ speedup is achieved by mapping the generalized summation $\sum_{j=2}^{i-1}[\ldots]$ from Equation (12) to a parallel prefix network. This network computes all final mixed-radix coefficients simultaneously.

1) Network Topology Selection: Kogge-Stone Network: The Kogge-Stone network is selected for its highly regular, pipelinable structure and optimal fan-out. This topology minimizes logic delays and is well-suited for high-frequency **ASIC implementation** by reducing routing complexity and clock skew. The network consists of exactly $\lceil \log_2 k \rceil$ uniform stages, where k is the number of moduli. For an FHE parameter set with $k = 50$ moduli, the sequential delay stages are reduced from $k = 50$ to just $\lceil \log_2 50 \rceil = 6$ stages, providing a factor of $\approx 8x$ speedup.

2) The Associative Combiner Module Design: The fundamental operation at each node of the Kogge-Stone network is the **associative combiner module**. This module merges two adjacent partial prefix results, (V_L, P_L) and (V_R, P_R), into a new combined result (V_{new}, P_{new}) that satisfies the associative property. The specific modular arithmetic operation is defined by the **Associative Merge Rule** (as defined in Section IV). The internal logic of this combiner module, derived from that rule, is composed of a single **Modular Multiplier** and a single **Modular Adder** to execute the two main terms in the V_{new} calculation. This composition ensures a uniform delay profile, Δ_{Comb}, across all $\lceil \log_2 k \rceil$ network layers.

C. Critical Path Latency and Area-Time Trade-off

The architectural realization of the Parallel MRC algorithm introduces a direct trade-off between hardware area (cost) and latency (speed).

1) Critical Path Delay and Latency: The total latency of the conversion, T_{total}, is dominated by the propagation delay through the $\lceil \log_2 k \rceil$ layers of the prefix network. The delay of a single Combiner Module, Δ_{Comb}, is primarily the delay of the largest Modular Multiplier within the module. The total conversion delay is definitively approximated by:

$$T_{\text{total}} = T_{\text{independent}} + T_{\text{prefix}} \approx O(1) + \lceil \log_2 k \rceil \cdot \Delta_{\text{Comb}}$$

This results in a definitive $\boldsymbol{T}(\text{MRC}) = \boldsymbol{O(\log k)}$ time complexity.

2) Area (Hardware Cost) Complexity: The hardware cost, or area complexity, is determined by the total number of Processing Elements and Combiner Modules.

- **Area of PEs:** The initial independent calculation phase requires $O(k)$ dedicated Processing Elements.

- **Area of Prefix Network:** The Kogge-Stone network requires approximately $O(k \log k)$ Combiner Modules.

The total area complexity is $A_{\text{total}} \approx O(k \log k)$. This trade-off is widely accepted in high-performance computing, positioning the Parallel MRC as the superior choice for high-throughput FHE accelerators.

VI. Software Library Architecture and Pre-computation

The algebraic decoupling of the Mixed-Radix Conversion (MRC) problem allows for a highly optimized software implementation, particularly suitable for deployment within cryptographic libraries (e.g., as an optimized function replacing the sequential MRC in FHE bootstrapping routines). This section details the necessary memory architecture and the two-phase execution model required to realize the $O(\log k)$ speedup in a software environment.

A. Pre-computation of Modular Inverses

The efficiency of the Dependency-Free Parallel MRC Algorithm relies fundamentally on moving the complexity from the runtime execution path to a **one-time pre-computation** phase. The primary computational bottleneck that is shifted is the calculation and storage of all required modular inverses.

- **Required Inverses:** The parallel identity requires two main types of pre-computed constants: the chain product of inverse moduli $Q_{l=1}^{i-1} m_l^{-1} \pmod{m_i}$ for the initial terms, and the partial product terms $N_{j,i} = Q_{l=j}^{i-1} m_l^{-1}$ for the parallel merge logic.
- **Complexity and Storage:** The pre-computation of these constants requires an $O(k^2)$ setup complexity, but this calculation is performed only once per RNS basis and is stored in a lookup table. The storage requirement for the lookup table is also $O(k^2)$ space.
- **Runtime Benefit:** Once stored, the MRC runtime becomes independent of this $O(k^2)$ factor, relying only on simple lookups and the $O(\log k)$ arithmetic. A dedicated helper function (such as an Extended Euclidean Algorithm routine) is used to ensure high-precision, robust calculation of these constants.

The explicit separation of the costly $O(k^2)$ setup phase from the quick $O(\log k)$ runtime phase is the core architectural tenet that enables high-throughput cryptographic operations in FHE libraries.

B. Two-Phase Parallel Execution Model

A software library implementation of the Parallel MRC is structured into two distinct phases, designed to utilize multi-core processors efficiently via thread-level parallelism (e.g., **OpenMP** or **CUDA**).

1) Phase 1: Parallel Coefficient Calculation ($O(1)$): This phase computes the initial independent terms P_i required for the subsequent parallel summation. Since the coefficients are now dependency-free, this entire phase is perfectly parallelizable, achieving an $O(1)$ latency when k threads are available.

The pseudocode for the parallel execution model for Phase 1 is:

```
// Phase 1: O(1) Coefficient Generation
IN PARALLEL FOR i = 1 TO k DO
    N_1_i <- LOOKUP(1, i)
    P_i   <- (x_i - x_1) * N_1_i MOD m_i
END FOR
```

2) Phase 2: Logarithmic Parallel Prefix Merge ($O(\log k)$): This phase uses the initial terms P_i to perform the final parallel prefix summation, directly mapping the associative rule to a logarithmic-depth structure.

The pseudocode for the logarithmic merge structure for Phase 2 is:

```
// Phase 2: O(log k) Prefix Merge
FOR d = 1 TO  log(k)  DO
    step_size <- 2^(d-1)
    IN PARALLEL FOR i = (2^d) TO k DO
        j <- i - step_size
        N_j_i <- LOOKUP(j, i)
        P_i <- P_i - (P_j * N_j_i) MOD m_i
    END FOR
END FOR
```

C. Integration into FHE Libraries

For use in a full FHE library, the $O(k^2)$ pre-computation step is ideally executed only once during the library's initialization phase. By moving the complexity off the main execution path, the library can execute the actual Mixed-Radix Conversion routine repeatedly during expensive operations like bootstrapping with guaranteed $O(\log k)$ latency, ensuring maximum throughput.

VII. Conclusion

This manuscript successfully resolves a critical scalability barrier for Fully Homomorphic Encryption (FHE) by introducing a Dependency-Free Parallel Mixed-Radix Conversion (MRC) Algorithm. Our approach fundamentally re-derives the sequential MRC equations, algebraically eliminating all data dependencies, which transforms the problem from a cascading sequence into an independent summation structure.

A. Summary of Significance

The mathematical soundness of the new algorithm was formally established via induction, proving its correctness across all moduli $k \geq 2$. Critically, the inherent parallelism was shown to reduce the time complexity from the traditional $O(k^2)$ to a provably optimal $O(\log k)$. This is the core achievement of this work, moving the MRC conversion from a dominant bottleneck to a highly efficient logarithmic operation.

B. Architectural Impact

The algorithmic breakthrough directly translates into a concrete, high-performance hardware architecture. The $O(\log k)$

complexity is realized through a direct mapping to a uniform $\lceil \log_2 k \rceil$-layer Kogge-Stone Parallel Prefix Sum network. The resulting architecture is entirely pipelinable and significantly reduces the circuit's critical path delay by replacing long sequential chains with a shallow logarithmic-depth network. This architectural efficiency makes the Parallel MRC solution immediately suitable for deployment in high-throughput FHE accelerators based on both ASIC and FPGA platforms.

C. Future Work

Future research stemming from this work will focus on two primary directions. The first is the exploration of an area-optimized variant of the prefix network (e.g., a hybrid Brent-Kung/Kogge-Stone topology) to further reduce the $O(k \log k)$ area overhead while maintaining high performance. The second is the development of a complete, end-to-end **VHDL/Verilog** implementation of the proposed architecture, coupled with silicon validation against standard FHE parameter sets, to provide precise power, performance, and area (PPA) benchmarks for commercial FHE services. By providing a provably efficient and scalable method for MRC, this work represents a fundamental step toward enabling the real-time, commercial deployment of FHE solutions.

Acknowledgments

The author would also like to acknowledge the assistance of a large language model, Gemini, for its role as a formalization and editing tool in the preparation of this manuscript. The author was the sole creator of the proof's logical structure. The AI was used under the direct control of the author to assist with refining the formal language, editing for clarity and conciseness, and ensuring proper formatting. All intellectual and creative decisions, as well as final editorial responsibility, rest with the author.

References

[1] C. Gentry, "A fully homomorphic encryption scheme," PhD Thesis, Stanford University, 2009.

[2] H. L. Garner, "The residue number system," *IRE Transactions on Electronic Computers*, vol. 8, no. 2, pp. 140–147, 1959.

[3] S. O. Im and S. I. Chae, "Improved mixed-radix conversion for residue number system architectures," *IEE Proceedings G (Circuits, Devices and Systems)*, vol. 138, no. 1, pp. 1–5, 1991.

[4] C. Ding, D. Pei, and A. Salomaa, *Chinese Remainder Theorem: Applications in Computing, Coding, and Cryptography*. Singapore: World Scientific Publishing Co., Inc., 1996.

[5] G. E. Blelloch, "Prefix sums and their applications," Carnegie Mellon University, Pittsburgh, PA, Tech. Rep. CMU-CS-93-200, 1993.

[6] P. M. Kogge and H. S. Stone, "A parallel algorithm for the efficient solution of a general class of recurrence equations," *IEEE Transactions on Computers*, vol. C-22, no. 8, pp. 786–793, 1973.